

Description:

Developed a Python script to automate the detection and tracking of suspicious IP addresses involved in high-frequency reservation activity on a peer-to-peer platform. This project simulates how a SOC team might monitor, flag, and respond to bot-like behavior that may contribute to reservation scalping or abuse.

Key Features:

- Parses web server logs (e.g., NGINX or Apache) to extract IPs, timestamps, and request patterns
- Identifies IPs that exceed a defined request threshold within a short time (for example, >30 requests in 1 minute)
- Maintains an **allowlist**, **watchlist**, and **blocklist** for IP addresses
- Outputs flagged IPs to a report file for manual review or further automated response
- Basic log of actions taken (flagged, ignored, or blocked)

Tools & Skills Used:

- Python
- Log analysis (regex, timestamps, request parsing)
- CSV/JSON file handling for storing watchlist/blocklist
- Security concepts: behavior-based detection, least privilege, IP filtering

IP Watchlist Automation for Reservation Abuse Detection

Overview

This project simulates a basic Security Operations Center (SOC) workflow by analyzing web server logs to detect high-frequency requests that may indicate reservation abuse (e.g., bots scalping restaurant reservations). It flags suspicious IP addresses and categorizes them into a watchlist or blocklist.

Features

- Parses access logs (NGINX/Apache-style) to extract IPs and request timestamps
- Detects IPs making more than 30 requests in 60 seconds
- Outputs flagged IPs into a `watchlist.txt` for SOC review
- Adds highly aggressive IPs to a `blocklist.txt`
- Basic behavior-based detection to help mitigate automated abuse

Use Case

This tool was inspired by a project at Cita Marketplace,

How to Use

1. Place your web server's `access.log` file in the same directory.
2. Run the script:

```
``bash
python ip_watchlist.py
```

CODE

```
import re
import datetime
from collections import defaultdict

# Thresholds
REQUEST_LIMIT = 30
TIME_WINDOW_SECONDS = 60

# Files
LOG_FILE = 'access.log'
WATCHLIST_FILE = 'watchlist.txt'
BLOCKLIST_FILE = 'blocklist.txt'

# Function to parse log line and extract IP and timestamp
def parse_log_line(line):
    match = re.match(r'^(\S+) - - \[(.*?)]', line)
    if match:
        ip = match.group(1)
        timestamp_str = match.group(2)
        timestamp = datetime.datetime.strptime(timestamp_str, '%d/%b/%Y:%H:%M:%S %z')
        return ip, timestamp
    return None, None

# Read log and build request timestamps by IP
requests_by_ip = defaultdict(list)
```

```

with open(LOG_FILE, 'r') as file:
    for line in file:
        ip, timestamp = parse_log_line(line)
        if ip and timestamp:
            requests_by_ip[ip].append(timestamp)

# Analyze IPs for suspicious activity
suspicious_ips = set()
for ip, timestamps in requests_by_ip.items():
    timestamps.sort()
    for i in range(len(timestamps)):
        count = 1
        for j in range(i+1, len(timestamps)):
            if (timestamps[j] - timestamps[i]).total_seconds() <= TIME_WINDOW_SECONDS:
                count += 1
            else:
                break
        if count > REQUEST_LIMIT:
            suspicious_ips.add(ip)
            break

# Write to watchlist and blocklist
with open(WATCHLIST_FILE, 'w') as watchlist, open(BLOCKLIST_FILE, 'w') as blocklist:
    for ip in suspicious_ips:
        watchlist.write(f'{ip}\n')
        if len(requests_by_ip[ip]) > 100: # Very aggressive IP
            blocklist.write(f'{ip}\n')

```

Project Summary :

IP Watchlist Automation

Built a Python-based log analysis tool to simulate SOC monitoring for a real-world scenario at Cita Marketplace. The script detects suspicious IPs making high-frequency web requests a common tactic used in reservation scalping. It generates actionable watchlists and blocklists to help mitigate automated abuse, reinforcing incident response and access control fundamentals.

Skills: Python, Log Analysis, Threat Detection, Security Automation, SIEM Concepts